

The one-night cram sheet.

Everything on this sheet was validated against the real contest assistant (Gemma 4 E4B, chat.ioai2026.kz) across three live rounds: Jul 29, Jul 30, and Aug 2 on your account. Memorize pages 1–2; everything else is reference.

THE THREE MAGIC STRINGS – ON EVERY CODE PROMPT

SUFFIX · ENDS EVERY PROMPT

Reply with **ONLY** a single python code block – no explanation, no reasoning, no draft versions. Max 30 lines, no comments.

OPENER · WHEN THE ASK NEEDS THINKING

Answer immediately with code – do not deliberate, do not analyze, do not restate the context.

NO-MOCK LINE · ALWAYS

Assume all named variables and data already exist – do NOT create mock, example, or placeholder data.

THE 7-STEP PLAY – EVERY PROBLEM, BOTH DAYS

#	STEP
1	Metric first. Find the scoring cell before reading the story. Open the problem's JupyterLab (Server button).
2	New chat in the assistant. One chat = one problem. Feed it the statement + baseline with P1 (parts <2,000 chars, "Reply only Noted.").
3	Submit the untouched baseline inside 30 min. Run All Cells → Git: stage → Commit → Push → Check. Points on the board, format verified.
4	Name the task family (tables overleaf) → paste its recipe into P3. The family recipe outranks whatever the model suggests in P2.
5	ONE change at a time. Run All Cells → compare self-score → keep if better, revert if not. Crash → P4 with the full traceback. Cut off → P6.
6	Two failed fixes → fresh chat , fully prescriptive re-ask (exact shapes, exact order, "no comments"). Never argue with a confused model.
7	Freeze 45 min early. P7 for imports/format, then YOUR eyes on the test-data check. Final commit → push → Check → score visible.

THE 5 RECOVERY MOVES

SYMPTOM	MOVE
Code cut off mid-line	"Continue EXACTLY from this line, code only:" + paste the last complete line (P6)
Empty reply	Resend once → still empty: add the Opener + "max 20 lines" → still: build it in pieces
Two code blocks in one reply	Always copy the LAST block — it drafts, then finalizes
Crash	P4: paste the cell + the ENTIRE red traceback. Fixes deprecated APIs first try
Fix failed twice	Fresh chat + prescriptive spec. Verified Aug 2: rescued a double-failure instantly

HARD RULES – EACH ONE IS A REAL SCAR

- Always paste, never reference. "The same code" → the model hallucinated a different problem entirely, even inside its 10-message window.
- Never modify test data. The shipped baseline's planted trap. And verified Aug 2: P7 said PASS while quoting the line that rotated the test images — this check is yours alone, 30 seconds, grep for rotate/noise/augment/random near "test".
- Check every number you specified appears in the reply. It silently turned 6 channels into 1 with a comment "assuming 1 channel".
- Delete the stray "python" first line when copying from the chat UI, and any "mock/example" lines.
- Run All Cells after every change. Partial-cell runs cause phantom NameErrors.
- Runs-but-wrong exists. Aug 2: a reshape that scrambled the channel axis ran fine and scored 0.35 instead of 0.999. The self-score comparison is the only defense — revert anything that doesn't improve it.
- Don't chase Scoreboard A. Medals come from hidden set B; simple robust changes transfer, leaderboard hacks don't.

ASSISTANT LIMITS (YOUR ACCOUNT, VERIFIED AUG 2)

LIMIT	VALUE	CONSEQUENCE
Replies	~2,000 tokens	one short cell per ask; caps are soft — read before running
Your message	2,000 chars	paste statements in parts ("Reply only Noted.")
Memory	last 10 messages	re-paste code every time; never say "the code above"
Rate	60 msg/hour	≈20 per problem; a full improve-loop costs ~2 — discipline, not quota, is the constraint

FREE WINS – DAY 1 FAMILIES (LIVE-VERIFIED JUL 29–30)

YOU SEE	YOU DO
Continual learning (old + new classes, $\frac{1}{2}$ ·old + $\frac{1}{2}$ ·new)	Expand head: new Linear(768, old+new), copy old weight rows in + replay: pd.concat a slice of ORIGINAL train.csv into fine-tune data. Never fine-tune on new classes alone
Behavioral cloning on grids	CNN on the grid (keep ALL channels) + concat state vector; class weights 1/bincount; mask invalid-action logits to $-1e9$ in the loss. No BFS/A*
Interactive oracle / 20 questions	Precompute the full answer table in <code>__init__</code> ; ask the question splitting candidates $\sim 50/50$; never eliminate on one mismatch (soft scores)
Clustering, mixed scales	StandardScaler BEFORE KMeans(<code>n_init=10</code>). One line: ARI 0.05 $\rightarrow$ 0.98
Small images, sklearn only	Pixels /255 $\rightarrow$ SVC(kernel="rbf", C=10). Never augment tiny images
Match two embedding sets 1:1	L2-normalize $\rightarrow$ cosine matrix $\rightarrow$ linear_sum_assignment(–sims). Never greedy argmax
Semi-supervised tabular	StandardScaler fit on labeled rows $\rightarrow$ LogisticRegression $\rightarrow$ predict unknowns in original order
Char sequence labeling	Context-window features, ONE dict key per offset (c-3...c+3, "#" out of range) $\rightarrow$ DictVectorizer $\rightarrow$ LogReg. List-valued key = silent F1 0.66

DAY-2 WILDCARDS (LIVE-VERIFIED AUG 2, YOUR ACCOUNT)

YOU SEE	YOU DO
Text classification / AI-text detection	TfidfVectorizer(analyzer="char", ngram_range=(2,5)) $\rightarrow$ LogisticRegression. Char n-grams beat word n-grams on short/noisy text
Broken model / weight repair	Compare state_dict tensors vs the healthy checkpoint (mean abs diff) $\rightarrow$ copy ONLY the outlier module. $\triangle$ Reject any load_state_dict(strict=False) on the full dict — it silently overwrites everything
Segmentation, classical	Per-pixel features: transpose(0,2,3,1).reshape(-1,C) — channel axis LAST — subsample 200k pixels $\rightarrow$ LogisticRegression. Wrong reshape runs fine and scores 0.35 vs 0.999
Time series regression	Lag-1/lag-2/rolling-mean-5 per signal $\rightarrow$ Ridge. Split by TIME, never shuffle
Feature engineering, fixed model	Row/col means+std+min+max + top-3 SVD singular values per sample $\rightarrow$ StandardScaler $\rightarrow$ the fixed model. Prescribe exact vector lengths or it garbles the concat
Multimodal fusion	Late fusion only: scale tabular $\rightarrow$ np.hstack with embeddings $\rightarrow$ LogisticRegression. Never joint deep training in a 20-min budget

THE SUBMIT RITUAL

Run All Cells $\rightarrow$ output file exists $\rightarrow$ Git tab $\rightarrow$ stage solution.ipynb $\rightarrow$ Commit $\rightarrow$ Push $\rightarrow$ contest page $\rightarrow$ Check $\rightarrow$ wait 5–9 min $\rightarrow$ score under Submissions. Grader re-runs the NOTEBOOK on hidden data: code is graded, not your local file. Never assume answers.json exists in test_public/. No internet, no pip, allowed libraries only.

CONTEST-DAY PROCEDURES

- Aug 3, 07:30 Practice Round — obligatory. Confirm quotas, the chat UI, JupyterLab, how the final solution is designated. Request your own keyboard/mouse.
- Freeze or crash → hand up immediately. Lost ≥10 min → extra-time form RIGHT THEN, in-hall. Round ends before approval → STAY SEATED.
- Clarifications must be Yes/No; paste confusing statements to Gemma first.
- Notebook limits: ~20 min runtime, 60 submissions/task, 18 GB VRAM. Train small.
- Anything Gemma gives you is legal — heavy use is the intended GAITE playstyle.
- Beating the baseline at all puts you on the board — never leave a task at zero.

TONIGHT, IN ORDER (THEN SLEEP)

MIN	DO
0–20	Read this sheet twice. Write the 3 magic strings + P4 from memory until exact.
20–50	One dry drill on your account (~48 msgs left this hour, resets hourly): P1-feed a home task, get one P3 improvement, run it. Stop at 50 minutes.
50–60	Read the three home-task hub cards (Task 1 audio • Task 2 robot • Task 3 oracle) — Day 1 extends exactly these.
then	Pack, charge everything, sleep. A rested operator beats a crammed one — the sheet only works if you can execute it at 09:00.